Round 5: solver side — requirements, barriers, and three

pre-registered solver/coupling tests 2026-11-04 · follow-up to `../round4/README.md` (#3387) · predictions in `.resume/RESUME.md` (commits `72dde4fd1e`, H; then L and Y), written before their runs · analysis `SOLVERS.md` · generator `../gen.sing` (new arms L and Y) · input hashes `SYSTEMS.sha256` · logs `logs/` · records `rows.tsv` · graph `round5.svg` · PDF `report.pdf`

Standing: every result below is an unverified assertion [pcc:unverified]. The statement claims are PCC-20261104-3d6b6c (test H), PCC-20261104-785e08 (test L), PCC-20261104-5d4597 (test Y) and PCC-20261104-e899a0 (the derivation). None has a signed verdict.

Class: derived bounds plus stage diagnostics. No pipeline is priced and no leaderboard row changes. Engine: msolve 0.6.5, p = 786433, target k = 0.

Result

The question was whether a better *solver*, rather than a better encoding, could close the gap. None of the options does at 256 bits. `SOLVERS.md` has the derivations.

1. **Faster Macaulay is not enough (derived)** [pcc:unverified].
   - Beating rho at 2^256 needs $\omega \lesssim 1.3$ **and** a solving degree flat in the tower depth.
   - With the measured depth slope, even $\omega = 1$ gives a ratio of 1.35.
   - The recorded matrices have 50–1,464 nonzeros per row, so sparse (block Wiedemann) elimination has an effective exponent of 2.41–2.72. Sparsity does not supply the missing factor.
2. **Enumeration-type solvers cannot beat √p (proved counting bound).** This covers SAT/CDCL branching on separators, meet-in-the-middle, dynamic programming over the tree, and exact tensor networks.
   - The root's two partial-sum sets satisfy $|A|\cdot|B| \gtrsim p$, so one of them has at least $\sqrt{p}$ elements.
   - Batching across targets does not help.
   - The coupling's incidence matrix has rank at least $|A|/2$, so exact tensor contraction cannot compress the bond.
3. **Geometric resolution (Kronecker) has intermediate degree ≳ p^{1−1/m}** (heuristic count).
4. **Test H, branching on separator characters: a measured no-op.**
   - At m3 t4 both branches reproduce T's matrices exactly (D = 6, widest 7,077, in 3.6–3.7 s), each with quotient dimension 6.
   - Over $\bar{F}_p$ every X is $c\cdot z^2$, so characters, power residues and "bits" carry no algebraic information. Making F_p-rationality algebraic takes degree p.
5. **Test L, fewer couplings with a fat leaf (one symmetric triple plus one leaf, 1 separator): prediction falsified.**
   - $D_L \geq 8$ at t = 4 (cut off at 1,800 s in degree 8, widest ≥ 79,300) and at t = 5 (memory cap at 782 s in degree 8), against T's D = 7.
   - The triple's node equation $S_4(e; X_1)$ has total degree 8. The biquadratic S_3 node (degree 4) is the minimum, and the pair tree already uses it everywhere.
6. **Test Y, a coupling that is a function rather than a 2-to-2 correspondence (y-coordinates and the chord slope): no gain, and worse in t.**
   - Unsaturated, it picks up 2,060 solutions over $\bar{F}_p$ at t = 4 (D = 7, 30 s), against T's 3. Almost all come from the degenerate chart $x_1 = x_2$, where $\lambda\cdot 0 = 0$ leaves the slope unconstrained.
   - With the chart removed by $v\cdot(w_2 - w_1) = 1$ (arm Yv), t = 4 gives D = 6, the same as T. It is 5.4× wider (38,133 columns), 20× slower (76 s), and has quotient dimension 12, which is 3 × 2 orderings × 2 signs.
   - At t = 5, Yv selects degree 7 and is cut off at 1,800 s, while T has D = 6 in 512 s. The function coupling therefore has the *steeper* t-slope.

Reading [pcc:unverified]. Of the solver families, only Gröbner bases compress, because their D stays far below the separators' domain sizes. Their cost is set by two slopes: about +1.5 per summand and about +0.5 per tower level, both coming from the couplings. Encoding the couplings differently (lifting, eliminating, fattening, function form), branching on cheap separator properties, and speeding up the linear algebra each leave the slopes unchanged or make them worse. An improvement on this line has to come from a node relation of degree below 4, which x-only coordinates cannot give, or from outside the summation-polynomial framework.

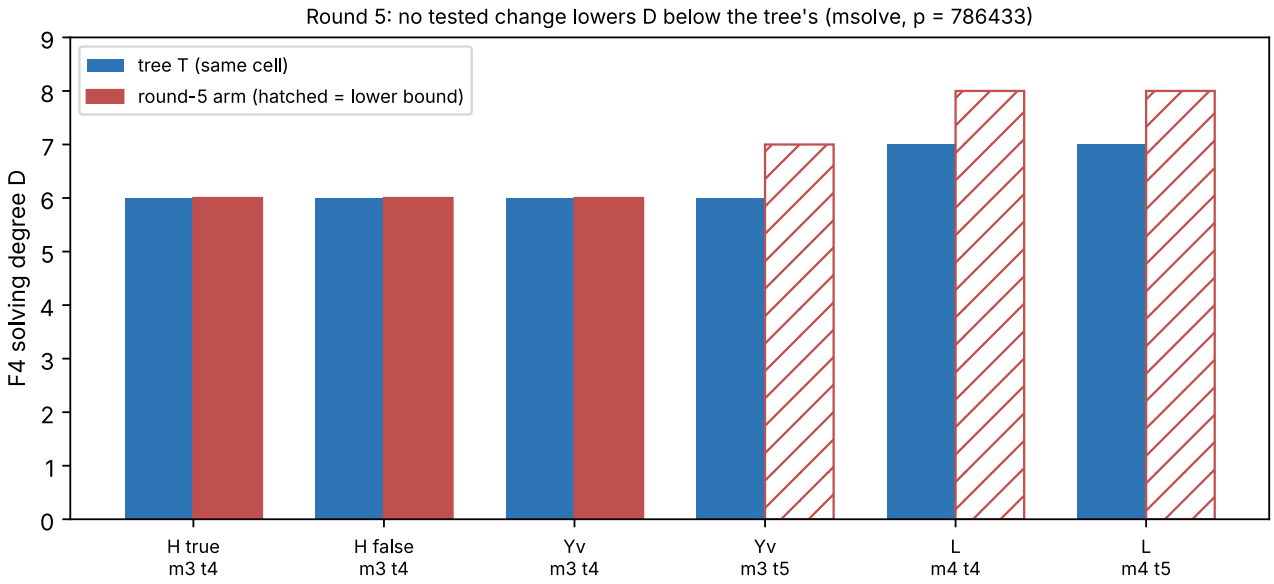


Table (recorded; `rows.tsv`)

cell	arm	N	D (≥: cut off)	widest	quotient dim	wall s	exit
m3 t4	T (round 1)	13	6	7,077	3	3.7	0
m3 t4	H true	14	6	7,077	6	3.70	0
m3 t4	H false	14	6	7,077	6	3.58	0
m3 t5	H true / false	17	6 (stopped)	19,852	—	42	143 stopped (redundant)
m3 t4	Y (unsaturated)	16	7	57,153	2,060	29.8	0
m3 t5	Y (unsaturated)	19	≥ 7	≥ 204,643	—	1,800	124
m3 t4	Yv	17	6	38,133	12	75.8	0
m3 t5	Yv	20	≥ 7	≥ 78,989	—	1,800	124
m3 t5	T (round 1)	16	6	23,720	3	512–544	0
m4 t4	L	14	≥ 8	≥ 79,300	—	1,800	124
m4 t5	L	18	≥ 8	≥ 278,857	—	782	139 memory (6 GiB)
m4 t4	T (round 1)	18	7	25,742	6	697	0

Two of the arms were built by text edits on generated systems:

- **H** appends z and $X_c - n \cdot z^2$.
- **Yv** appends v and $v \cdot (w_2 - w_1) - 1$.

Arms L and Y are generated by `gen.sing` with the planted check. The appended H and Yv equations hold at the planted point by construction: n was chosen from the planted X's character, and $w_1 \neq w_2$.

Caveats

- The H runs at m3 t5 and m4 t4 were stopped or not run once the t4 result showed the construction adds no algebraic constraint.
- Wall times come from runs with up to 2 msolve jobs on 4 cores, and should be read only as an ordering.
- The cut-off cells are lower bounds.

Round 5: the solver side — what a faster solver would have to do, and

why the obvious ones cannot 2026-11-04 · follow-up to `../round4/README.md` and [pcc:unverified] `../round4/CROSSOVER.md`. Question: rounds 1–4 show that no encoding change lowers the tree's cost. Is there a solver whose cost is not the dense Macaulay size: SAT, faster or sparse Macaulay linear algebra, tensor networks, geometric resolution, hybrids? Tests H and L were pre-registered in `.resume/RESUME.md` (commits `72dde4fd1e` and the next one) before their runs. **Class: derived bounds plus two stage diagnostics.** No pipeline is priced.

1. What "faster Macaulay" would have to achieve (derived)

This is the round-4 cost model, with the linear-algebra exponent ω swept from [pcc:unverified] the information floor (1, linear in matrix size) up. Cost is $\log_2$ cost / $\log_2 \sqrt{p}$ at the best m; below 1 beats rho.

ω	β = 0 (D flat in t)	β = 0.5 (measured t-slope)
1.0	2^128: 1.14 · **2^256: 0.86** · 2^512: 0.64	2^256: 1.35 · 2^512: 1.06 · 2^1024: 0.82
1.2	2^256: 0.95	2^256: 1.54
1.5	2^256: 1.04	2^256: 1.84
2.0	2^256: 1.22	2^256: 2.33

At 256 bits, beating rho needs **both** near-linear linear algebra ($\omega \lesssim 1.3$) **and** a solving degree flat in the tower depth ($\beta \approx 0$). With the measured depth slope, even $\omega = 1$, linear in the size of the matrix, does not get there. Then the size of the matrix itself is the obstacle, not the elimination.

Sparsity does not supply ω ≈ 1. The recorded msolve matrices are not sparse enough. Block Wiedemann costs about cols² · (nonzeros per row), which gives an effective exponent log(cols²·w)/log(cols) between 2.41 and 2.72:

matrix	widest	nonzeros per row	exponent
T at m4 t4	12,782 × 25,742	713	2.65
F at m4 t4	15,086 × 13,503	50	2.41
T at m3 t5	26,344 × 23,720	1,464	2.72
T at m5 t4	372,489 × 654,028	654	2.48

(These come from the logs' density column, by awk.)

2. Three barriers for solvers that avoid Macaulay matrices

(a) Enumeration-type solvers: SAT/CDCL branching, meet-in-the-middle, [pcc:unverified] dynamic programming over the tree, exact tensor-network contraction. *Counting bound (proved).* In any decomposition tree, the root's two children carry partial sums from sets A and B. Covering a random target R requires $R \in A + B$, hence $|A|\cdot|B| \gtrsim p$ up to the symmetry factor, and therefore $\max(|A|, |B|) \gtrsim \sqrt{p}$.

A solver that decides a separator by its value has to range over at least one of these sets. Branching on its bits, tabulating it, or contracting a bond indexed by it all count. So it pays at least $\sqrt{p}$, the same as rho, and batching over targets does not escape this. Here is the batching calculation. A table of k-fold sums costs |V|^k. Each target scans |V|^{m−k}. There are about |V|·p/|V| ^m targets per |F| relations. Keeping both terms below $\sqrt{p}$ would require $|V|^k \le \sqrt{p}$ and $|V|^{k-1} \ge \sqrt{p}$, which contradict each other. These solvers are generic in effect.

Rank version, for tensor networks. The root coupling $S_3(X_1, X_2, x_R) = 0$ is a 2-to-2 correspondence, a curve isomorphic to E. Its incidence matrix on A × B has at most 2 ones in each row and column, so its rank is at least |A|/2. No change of basis compresses the bond, which settles exact contraction. Approximate low-rank contraction would need a different argument.

(b) Geometric resolution, i.e. the Kronecker solver (cost ≈ δ², with δ the largest intermediate degree). If the membership equations come before the coupling, the variety cut out by all m membership constraints already has |V|^m ≈ p points. In the other order, the coupling variety with k leaves restricted has degree ≈ |V|^k · c, so $\delta \gtrsim p^{1-1/m}$ at the last step. Either way the cost is above p. *Heuristic count, with no ordering found that avoids it.*

(c) Character and rationality branching (test H, measured). The idea is to branch on a property of a separator cheaper than its value: its quadratic character, imposed as $X_1 = n \cdot z^2$ with $n \in \{1, \text{a non-square}\}$, one bit per branch.

- At m3 t4 both branches reproduce T exactly: D = 6, widest 7,077, the same sum of rows×cols, in 3.6–3.7 s.
- Each branch has quotient dimension 6, against T's 3, because $z \mapsto -z$.
- Gröbner bases count solutions over $\bar{F}_p$, where every X is $c \cdot z^2$. So "X ∈ F_p is a square" carries no algebraic information. Making F_p-rationality algebraic takes degree p (z^p = z).

- The same holds for every multiplicative character, power-residue class, or "bit" of a separator: algebraic solvers cannot use them.
- The m3 t5 branches tracked T's matrices exactly until they were stopped at 42 s. m4 t4 was not run.

3. The one structural lever left, and test L

Fixing the separators gives D = 4 in every cell (rounds 2–3). So the excess degree is the cost of the couplings. A natural solver-structure question is whether *fewer* couplings give a smaller excess.

Test L, the fat leaf, takes m = 4 as one triple plus one leaf: the triple is encoded symmetrically (S_4 in e_1, e_2, e_3 with output X_1), giving 1 separator instead of the pair tree's 2.

- **Prediction falsified:** D_L ≥ 8 at t = 4 and 5, against D_T = 7.
- **Cause:** the triple's node equation has total degree 8, against 4 for the pair tree's S_3-type nodes, so the input degree alone gives D ≥ 8.
- **Lesson:** the excess comes from the number of couplings times the degree of each node relation. S_3's biquadratic, degree 4, is the floor for an x-only addition relation, and the pair tree already uses it everywhere.
- The final L numbers are in `README.md`.

4. Where that leaves the solver side

solver family	cost driver	verdict at these laws
Gröbner/F4, dense or sparse	Macaulay size C(N + D, D), with N ≈ $\log_2 p$	asymptotically subexponential if D = O(m), D flat in t and ω ≲ 1.3. Measured: the sparse exponent is 2.4–2.7 and the t-slope is about 0.5
SAT, MITM, DP, exact tensor networks	separator domain size	≥ √p (proved counting bound)
geometric resolution (Kronecker)	intermediate degree δ	≳ p^{1−1/m} (heuristic)
hybrid on separator characters	none: invisible over $\bar{F}_p$	measured no-op

Reading. At 256 bits every solver family is held at or above rho by a different mechanism. Compression is possible only in Gröbner, where it shows up as D being far below the separators' domain sizes. Its price is the Macaulay size, which grows because each added summand adds about 1.5 to D and each tower level about 0.5.

So a real improvement has to change those two slopes. The candidates are: [pcc:unverified]

- a node relation of degree < 4, which x-only coordinates cannot give;
- a coupling that is a function rather than a 2-to-2 correspondence (y-coordinate or slope encodings). Test Y measured it afterwards: same D at t = 4, a steeper t-slope, and 5–20× the cost (`README.md`);
- leaving the summation-polynomial framework altogether.

No idea tried in rounds 1–5 changes either slope.